

Hold My Pipette, I'm Doing De Novo Assembly Using Oxford Nanopore Reads

18th September 2025 – MRC GAMBIA

Leonard N., Shola A., Dam K., Noutin M., Kanny D.

Content

- ❖ Gain an understanding of what can be accessed in the MinKNOW software during and after an ONT run, including file outputs
- ❖ Understand how to turn raw Oxford Nanopore (ONT) raw reads into an assembly on CL.
- ❖ Step-by-step overview of the pipeline (trimming → QC → assembly → assembly evaluation).

On Tuesday: Run Configuration

Run configuration Load configuration

Experiment name	Wellcome_GROUP-1_16SEP25
Sample ID	Multi_Pathogten_ONT
Flow cell type	FLO-MIN114
Sequencing kit	Rapid Barcoding Kit 24 (SQK-RBK114-24)

Sequencing and analysis

Basecalling ☒ Edit

Model Super-accurate - v5.0.0 - 400bps

Modified bases Off

Barcoding ☒ Edit

Trim Off

Selected barcodes All

Alignment ☐

Adaptive sampling ☐

Advanced sequencing options Edit

Run limit Stop run when...

Output

Please note that during in the second half of 2025, raw data output (FAST5 and POD5 files) will no longer be enabled by default. If raw data is required, you will need to re-enable this option at run start-up. Edit

File output options

File location /data/ Edit

386.8 GB used - 6.2 TB free

Data format FASTQ

Output frequency Hourly

FASTQ compression On

Barcode splitting On

Raw reads POD5

Filtering ☒ Edit

Min Q score 10

ONT – Folder Structures

❖ Pass vs Fail: read quality filter

Based on Q-score (and sometimes length).

❖ **Pass** = above threshold; **Fail** = below. Independent of barcodes.

❖ Classified vs Unclassified:

Classified = a barcode was confidently detected

Unclassified = barcode not confidently detected

barcode_al...379472e5.tsv ✓	barcode01	FAY32108_p...5_0.fastq.gz	Aug 13, 2024 at 1:06 PM
fastq_fail	barcode02	FAY32108_p...5_1.fastq.gz	Aug 13, 2024 at 1:16 PM
fastq_pass	barcode03	FAY32108_p...5_2.fastq.gz	Aug 13, 2024 at 1:26 PM
final_summa...79472e5.txt ✓	barcode04	FAY32108_p...5_3.fastq.gz	Aug 13, 2024 at 1:36 PM
other_reports	barcode05	FAY32108_p...5_4.fastq.gz	Aug 13, 2024 at 1:46 PM
pod5	barcode06	FAY32108_p...5_5.fastq.gz	Aug 13, 2024 at 1:56 PM
pore_activity...9472e5.csv ✓	barcode07	FAY32108_p...5_6.fastq.gz	Aug 13, 2024 at 2:06 PM
report_FAY3...4b3362.html ✓	barcode08	FAY32108_p...5_7.fastq.gz	Aug 13, 2024 at 2:16 PM
report_FAY3...4b3362.json ✓	barcode09	FAY32108_p...5_8.fastq.gz	Aug 13, 2024 at 2:26 PM
report_FAY3...a4b3362.md	barcode10	FAY32108_p...5_9.fastq.gz	Aug 13, 2024 at 2:36 PM
sample_she...a4b3362.csv ✓	barcode11	FAY32108_p..._10.fastq.gz	Aug 13, 2024 at 2:46 PM
sequencing...379472e5.txt	barcode12	FAY32108_p...5_11.fastq.gz	Aug 13, 2024 at 2:56 PM
throughput_...79472e5.csv ✓	barcode13	FAY32108_p..._12.fastq.gz	Aug 13, 2024 at 3:06 PM
	barcode14	FAY32108_p..._13.fastq.gz	
	barcode15	FAY32108_p..._14.fastq.gz	

ONT – Folder Structures

❖ MinKNOW software chunks fastq files:

By read count – “start a new FASTQ every N reads”

By time - “start a new FASTQ every M minutes/hours”

```
├── barcode01/
│   ├── FAY68075_pass_barcode01_df6fcce_3e6be048_0.fastq.gz
│   ├── FAY68075_pass_barcode01_df6fcce_3e6be048_1.fastq.gz
│   ├── FAY68075_pass_barcode01_df6fcce_3e6be048_2.fastq.gz
│   └── ...
├── barcode02/
│   ├── FAY68075_pass_barcode02_df6fcce_3e6be048_0.fastq.gz
│   ├── FAY68075_pass_barcode02_df6fcce_3e6be048_1.fastq.gz
│   ├── FAY68075_pass_barcode02_df6fcce_3e6be048_2.fastq.gz
│   └── ...
├── barcode03/
│   ├── FAY68075_pass_barcode03_df6fcce_3e6be048_0.fastq.gz
│   ├── FAY68075_pass_barcode03_df6fcce_3e6be048_1.fastq.gz
│   ├── FAY68075_pass_barcode03_df6fcce_3e6be048_2.fastq.gz
│   └── ...
```

Output fastq file organization

- ❖ Reads are automatically demultiplexed during basecalling
- ❖ Fastq files are generated in real-time (every 10 minutes in this example), so multiple fastq files are generated for each barcode (sample)

Sequencing run QC using the “report_***.html” file

- ❖ Check that the amount of data generated matches your expectation (this is a function of library loading concentration, run time, number of pores available for sequencing)
- ❖ Typically, for a fresh flow cell with a run that's 48+ hours, ~20 Gbases of data are generated
- ❖ If the flow-cell is being re-used and has <800 pores left (like example below), data generated will be lower

^ Run summary

DATA OUTPUT

Estimated bases

3.09 Gb

Reads generated

3.63 M

Estimated N50

860 b

BASECALLING

Reads called

100%

Reads called (min Q score: 9)

3.15 M

480.34 k

Pass

Fail

Bases called (min Q score: 9)

2.44 Gb

347.16 Mb

Pass

Fail

RUN DURATION

Run time

24 hrs 0 mins / 24 hrs 0 mins

Elapsed time



Run limit

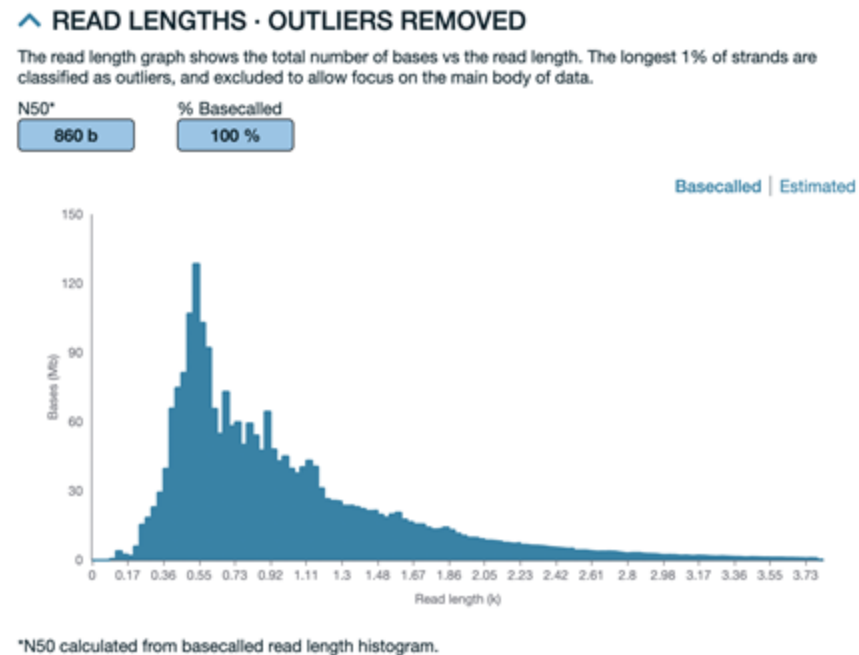


Run status

FINISHED · Target runtime has been reached

Read length distribution

- ❖ Check that the distribution of read lengths matches expectation (depends on quality of sample going in, whether fragmentation was performed, etc.)
- ❖ Distribution should roughly match what was seen in Tapestation (if used), although it's common for read lengths to skew shorter



Cumulative output as a function of time

- ❖ Cumulative output should start out linear and then plateau off as pores get exhausted
- ❖ >80% of bases should pass the Q-score threshold (usually set to 9)

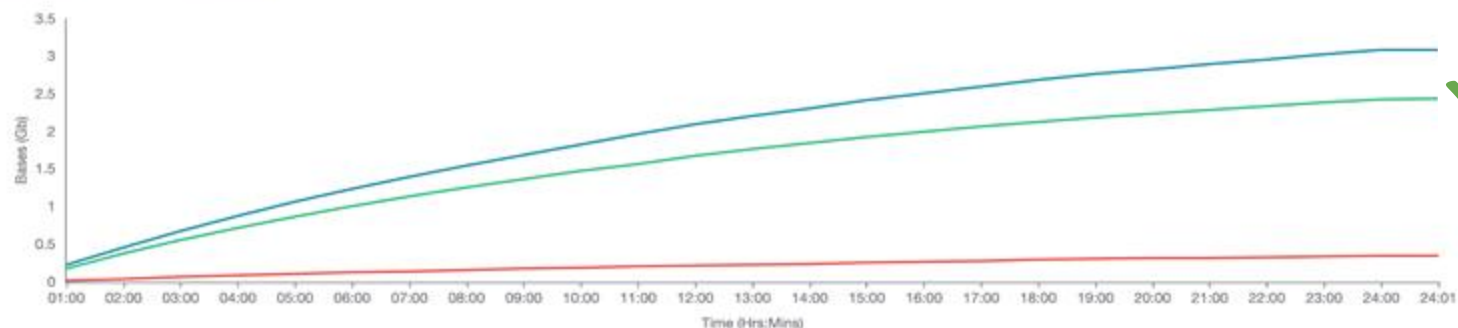
^ CUMULATIVE OUTPUT

The cumulative output shows the total amount of bases or reads sequenced over time by your device.

Bases

Legend

— Estimated	— Passed	— Failed
Predicted total number of bases, prior to basecalling	Bases equal to or above the quality score threshold.	Bases below the quality score threshold.



Reads

Legend

— Total	— Passed	— Failed	— Skipped
Total number of reads, including passed, failed and skipped.	Reads equal to or above the quality score threshold.	Reads below the quality score threshold.	Reads that will not be basecalled. Post run basecalling is possible.

Cumulative output as a function of time

- ❖ Cumulative output should start out linear and then plateau off as pores get exhausted
- ❖ >80% of bases should pass the Q-score threshold (usually set to 9)

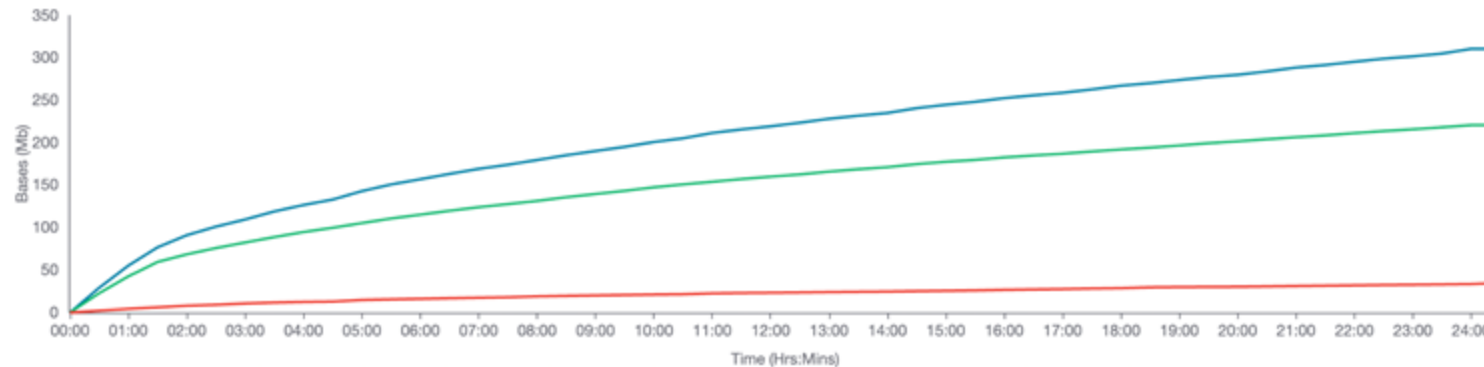
^ CUMULATIVE OUTPUT

The cumulative output shows the total amount of bases or reads sequenced over time by your device.

Bases

Legend

— Estimated	— Passed	— Failed
Predicted total number of bases, prior to basecalling	Bases equal to or above the quality score threshold.	Bases below the quality score threshold.



Cumulative output as a function of time

- Cumulative output should start out linear and then plateau off as pores get exhausted
- >80% of bases should pass the Q-score threshold (usually set to 9)

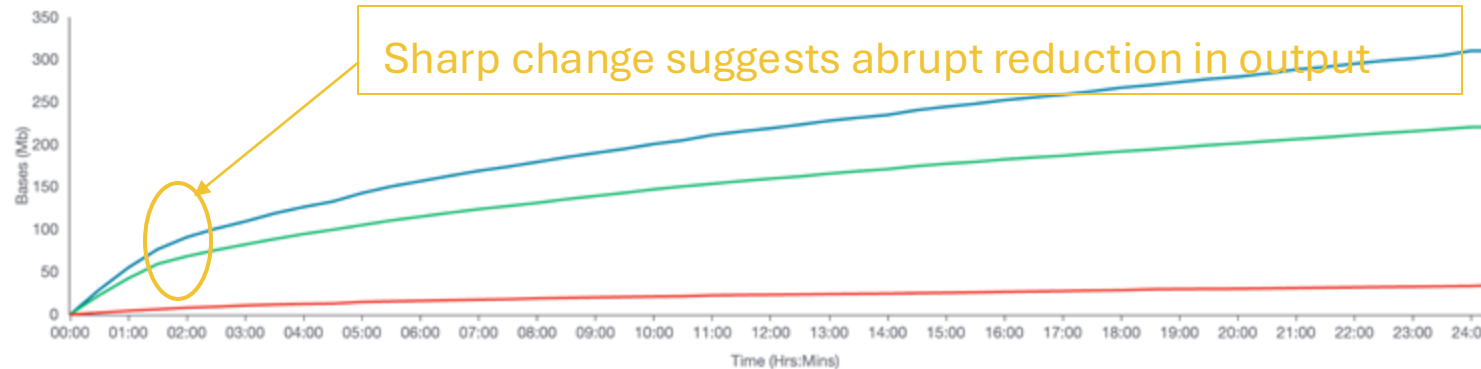
^ CUMULATIVE OUTPUT

The cumulative output shows the total amount of bases or reads sequenced over time by your device.

Bases

Legend

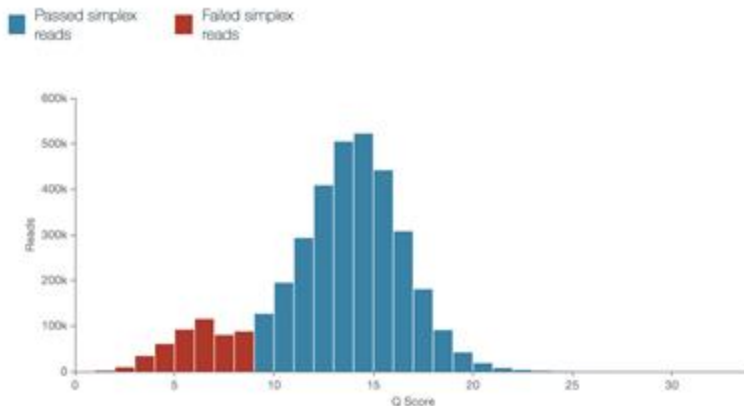
— Estimated	— Passed	— Failed
Predicted total number of bases, prior to basecalling	Bases equal to or above the quality score threshold.	Bases below the quality score threshold.



Q-score

- ❖ Q-scores are generally lower than what we get for Illumina; a mean score of 15 is typical
- ❖ Ideally, Q-scores will remain steady over time

Q Score histogram



Q Score over time

Legend

— Mode

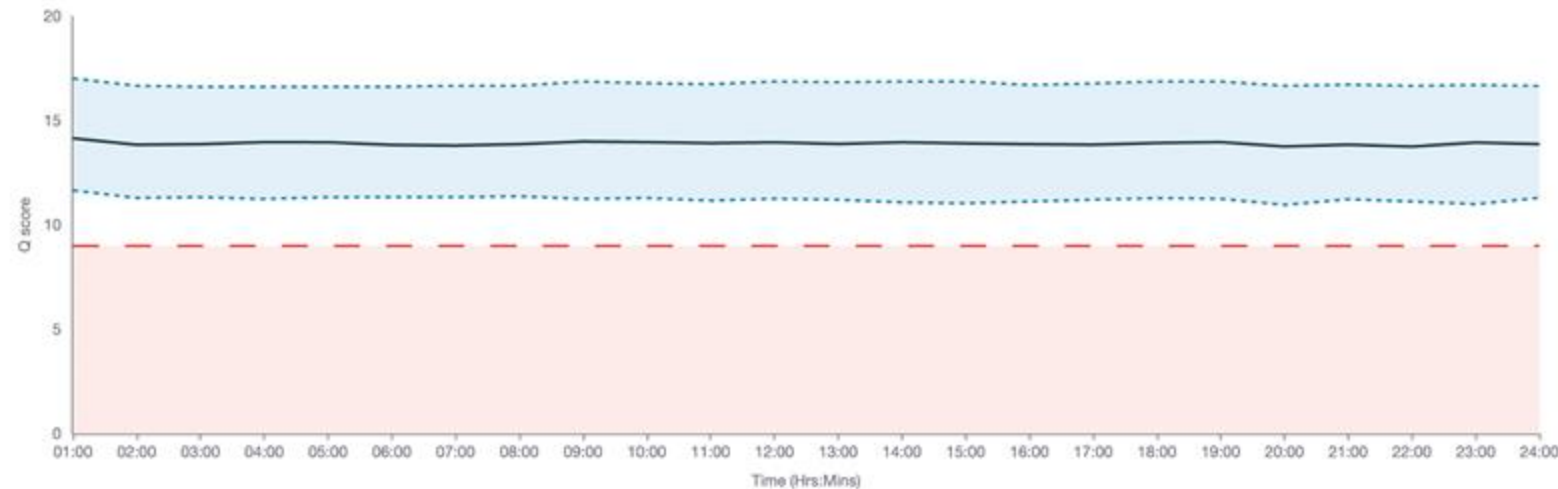
The most frequent quality score of reads in the run.

■ Spread

The spread of quality scores, found by calculating full width half maximum.

--- Min. quality score

Minimum quality score to be accepted as a passed read.



Q-score

- Q-scores are generally lower than what we get for Illumina; a mean score of 15 is typical
- Ideally, Q-scores will remain steady over time

QUALITY SCORE

The quality score is calculated as basecalling is performed on your device. Reads that fall below the minimum value of 9 will be classified as failed reads. You can alter the accepted minimum quality score in MinkNOW.

Legend

— Mode

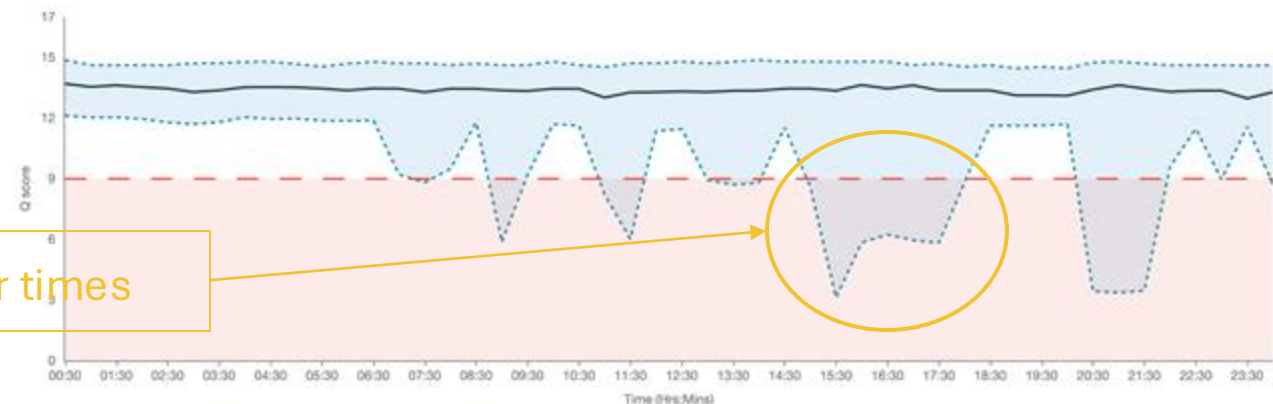
The most frequent quality score of reads in the run.

□ Spread

The spread of quality scores, found by calculating full width half maximum.

--- Min. quality score

Minimum quality score to be accepted as a passed read.



Greater variation (reduction) in Q-score at later times

Pore activity over time

- ❖ Sequencing/available pores gradually decrease over time, with slight increases when pore scans are performed

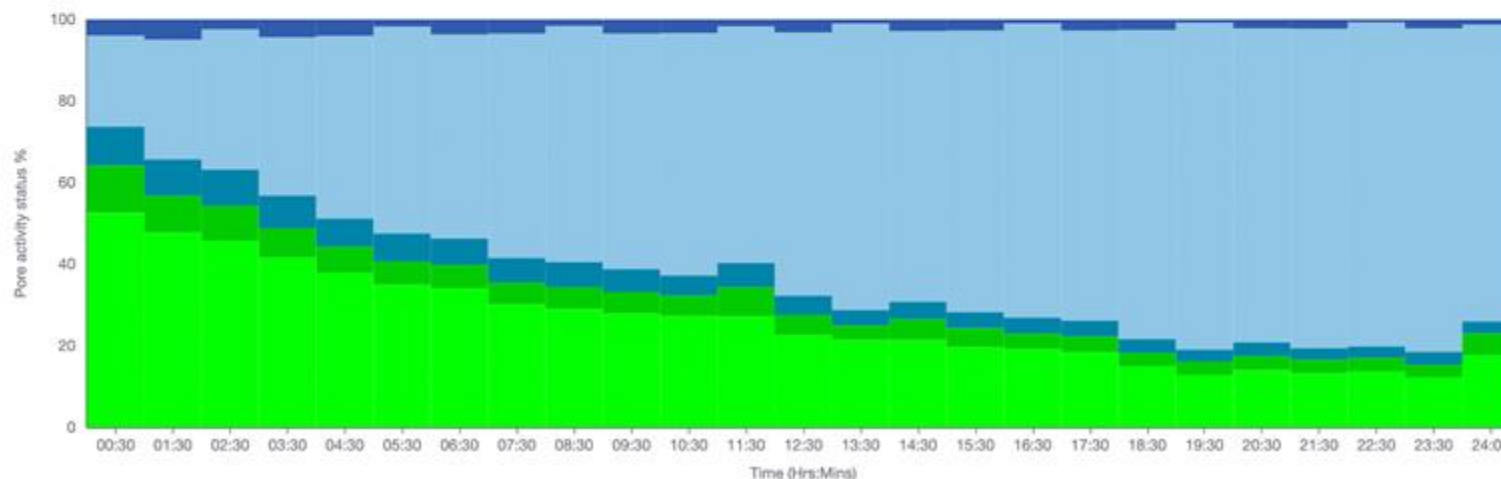
Run health

PORE ACTIVITY

The Pore activity graph shows the performance of your sample as it is being sequenced during a run.

☒ Show grouped

Legend



Pore activity over time

- ❖ See [here](#) for how to diagnose issues based on pore activity plots

PORE ACTIVITY

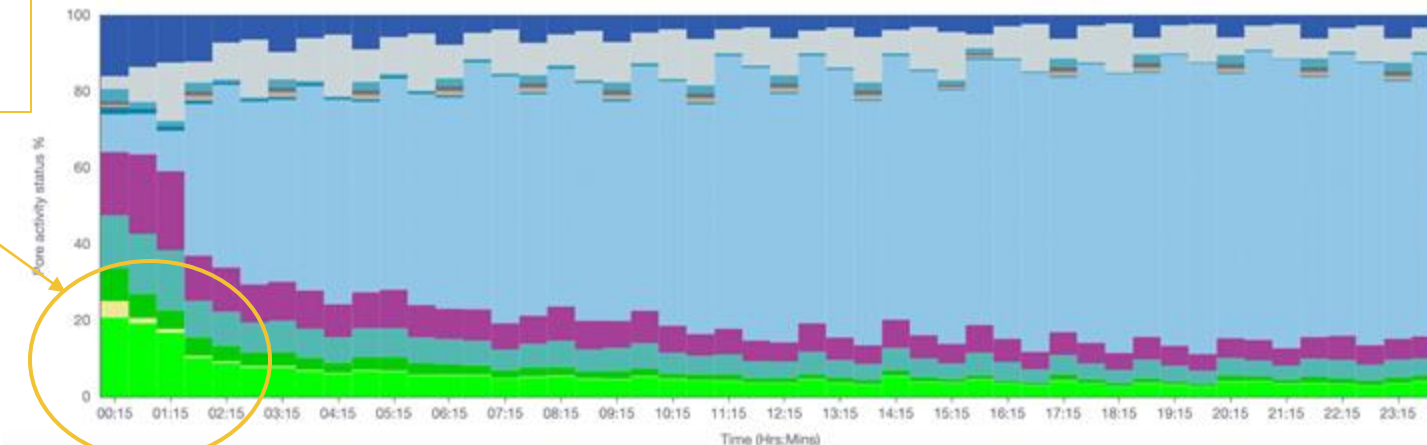
The Pore activity graph shows the performance of your sample as it is being sequenced during a run.

Legend



Lots of unusual pore statuses

Low starting % of active pores



Introduction – Command Line Analysis

Dr. Bioinformagician



Many tools:

qc
trimming,
assembly e.t.c.

Dependencies:

Tool A $\rightarrow$ (B, C)
B $\rightarrow$ (D, E_v1)
C $\rightarrow$ (E_v2, F)
E_v1 + E_v2 can't
interact.



What is needed (package manager):
Isolated environments per project
Pinned versions for exact rebuilds
Share a recipe, not a laptop



- Environment isolation
- Version Pinning
- Shareability with the bioinf. community

Downloading & Installing conda

Getting Started > Miniconda

Installing Miniconda

Access the URL:
<http://tiny.cc/gdns001>

conda install fastqc

AI Mode All Videos Images Short videos Forums Web More ▾ Tools ▾

Anaconda.org
<https://anaconda.org> > bioconda > fastqc

[Fastqc - bioconda](#)

osx-64 v0.11.8; linux-64 v0.11.8; noarch v0.12.1. **conda install**. To install this package run one of the following: **conda install bioconda::fastqc**

Windows Command Prompt Windows PowerShell macOS Linux

1. Run the following four commands to download and install the latest Linux installer for your chosen chip architecture. Line by line, these commands:
 - create a new directory named "miniconda3" in your home directory.
 - download the Linux Miniconda installation script for your chosen chip architecture and save the script as `miniconda.sh` in the miniconda3 directory.
 - run the `miniconda.sh` installation script in silent mode using bash.
 - remove the `miniconda.sh` installation script file after installation is complete.

```
64-bit AWS Graviton 2/ARM 64 IBM Z
mkdir -p ~/miniconda3
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O
bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
rm ~/miniconda3/miniconda.sh
```

conda install ?

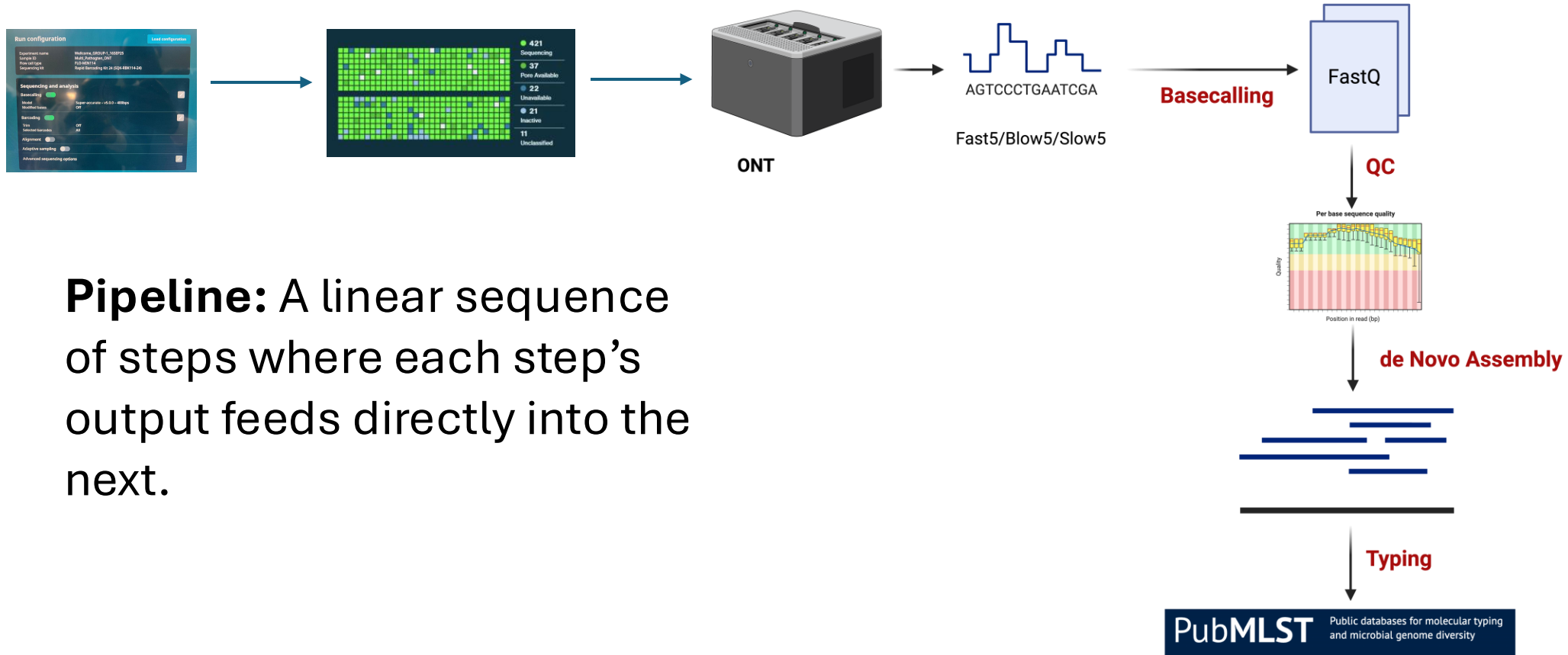
To install this package run one of the following:

```
conda install bioconda::fastqc
```

```
conda install bioconda/label/broken::fastqc
```

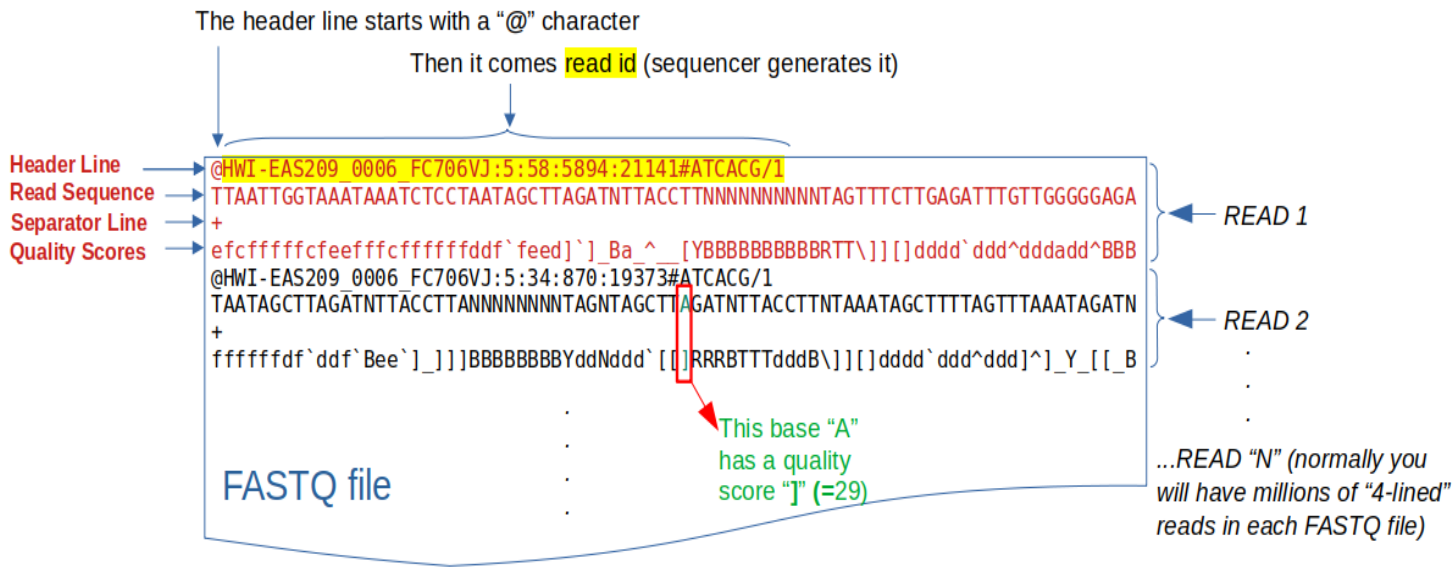
```
conda install bioconda/label/cf201901::fastqc
```

Overview of the Pipeline



FASTQ Format and Quality Score

fastq file



- Sequencing instruments assign each base a **Phred quality score (Q)**
- The score is logarithmic: meaning a **10-point increase** in Q is a **10-fold reduction** in error probability.

Phred Quality Score	Probability of incorrect base call	Base call accuracy
10	1 in 10	90%
20	1 in 100	99%
30	1 in 1000	99.9%
40	1 in 10000	99.99%
50	1 in 100000	99.999%

fasta file

```
>abcZ
TTTGATACCGTTGCCGAAGGTTTGGGTAAAATTCGCGATT
GTCGGTCATGAGTTGGAAAACGGTTCGGGTGAGGCTTTGT
CAACTTGAAATCGAAGCGAAGGACGGCTGGAAGCTGGATG
GGCGAACTCGGTTTGCCGGAACGAAAAATCGGCAACC
CGTGTCGCCTTGGCGCAGGCTTGGGTGCAGAAGCCCGACG
ACCAACCATTGATATTGACGCGATTATCTGGTTGGAAA
GGCAGCTTGGTCGTGATTACCCACGACCGCCGTTTTTTGG
GTCGAACTTGACC
```

ONT De Novo Assembly Pipeline

```
1  cat << EOF
2
3
4  #####
5  # Authors: Leonard N., Shola A., Noutin, Kanny D.
6  # Date: 16th September 2025
7  # Purpose: ONT De Novo Assembly Pipeline (ONT-only)
8  #           Checks dependencies, gives install commands
9  #####
10
11  usage() {
12  |   cat << 'EOF'
13  |   ONT De Novo Assembly Pipeline (ONT-only)
14  |   =====
15  |   Description:
16  |       - Concatenate barcode FASTQs
17  |       - Adapter trimming (Porechop)
18  |       - QC (FastQC, MultiQC)
19  |       - Filtering (NanoFilt)
20  |       - Assembly (Flye)
21  |       - Evaluation (QUAST)
```

Functions of commands



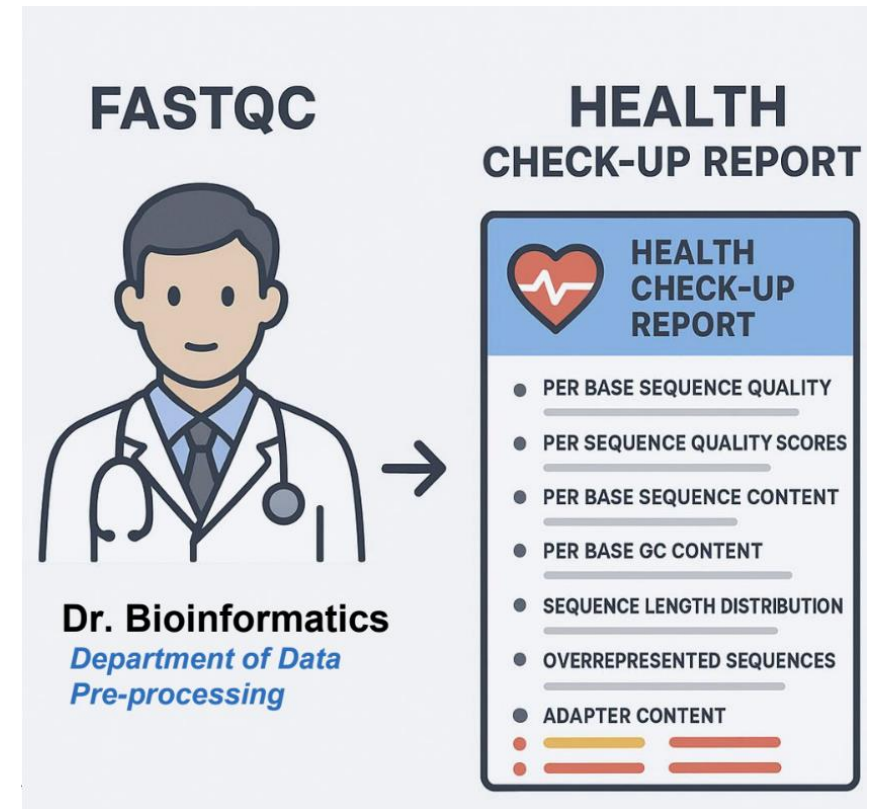
```
cat 16s_banjul.fasta 16s_abidjan.fasta > combined.fasta
```

To combine/concatenate files

QC

QC Modules Just like a Medical Check-Up

- ❖ **Per Base Sequence Quality** → Blood Pressure
Measures how healthy each position in the read is.
Sudden drops = a problem.
- ❖ **Per Sequence Quality Scores** → Overall Health Score
Are most reads healthy or are many poor quality?
- ❖ **Adapter Content** → Foreign Body Detection
Like detecting a parasite - these shouldn't be in your sample and need removal.
- ❖ **Per Base GC Content** → Metabolic Balance
Deviation from expected GC% hints at contamination or biased library prep.
- ❖ **Sequence Length Distribution** → Height Chart
Are your reads the expected length? Shorter or mixed sizes may indicate trimming or fragmentation issues.



Why Sequence Quality Control Matters?

- ❖ **Confirms data quantity/quality.** Do we have enough reads, long enough, and good enough?
- ❖ **Prevents wasted compute.** Don't spend hours assembling junk; QC first, assemble once.
- ❖ **Stops adapter/barcode bleed.** Trim barcodes/adapters; prevent misassignments before assembly

Catches contaminations early: through GC content assessment

Low-quality bases can lead to incorrect genome assemblies
or false variant calls

QC Tools



FastQC is a tool for diagnosis of your raw sequencing reads.
Provides per-sample reports (HTML + data files).

Key checks include:

- Sequence quality scores.
- Sequence length distribution.
- Adapter/overrepresented sequences.

Helps detect problems early (e.g., low-quality runs).

Usually run before and after trimming/filtering to compare improvements.



MultiQC is a tool that aggregates results from multiple samples and tools into one report

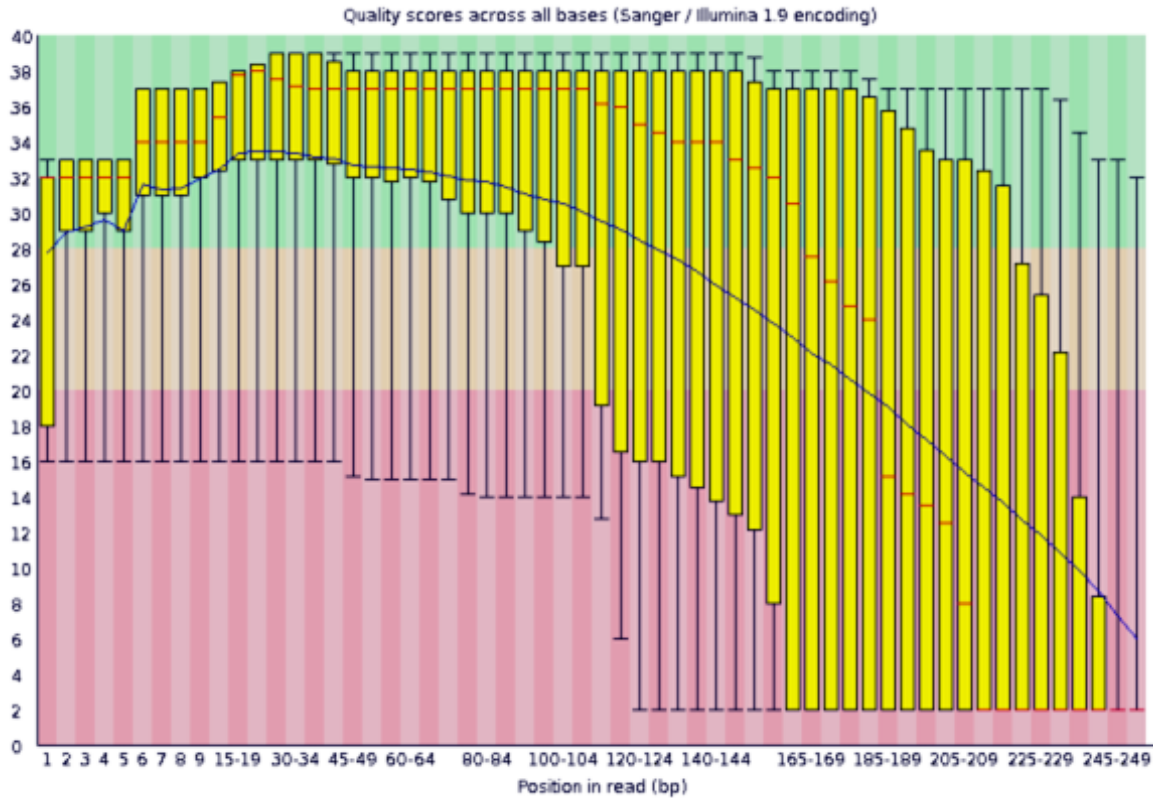
Instead of looking at 50 separate FastQC HTML files, you can run MultiQC on a folder and get a single interactive report summarizing all samples.

Other tools for QC: *fastp*, *SeqKit* e.t.c

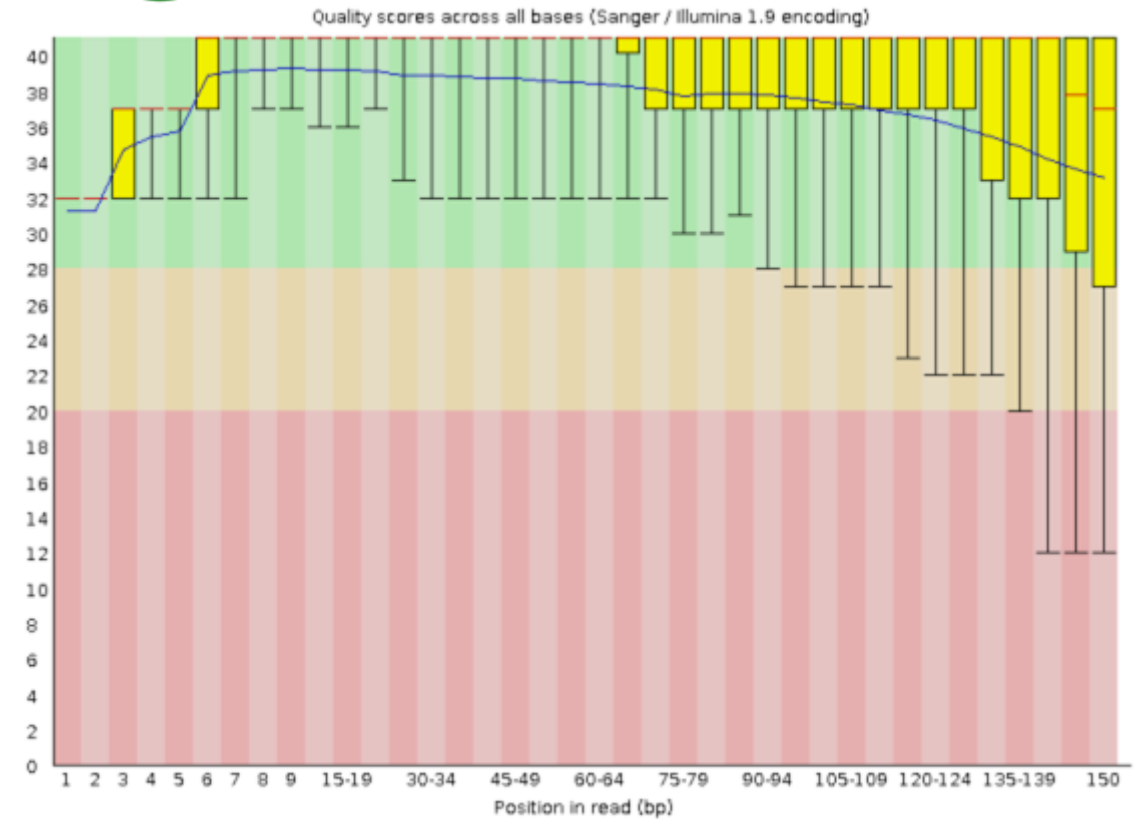
Per Base Sequence Quality



BAD per base quality

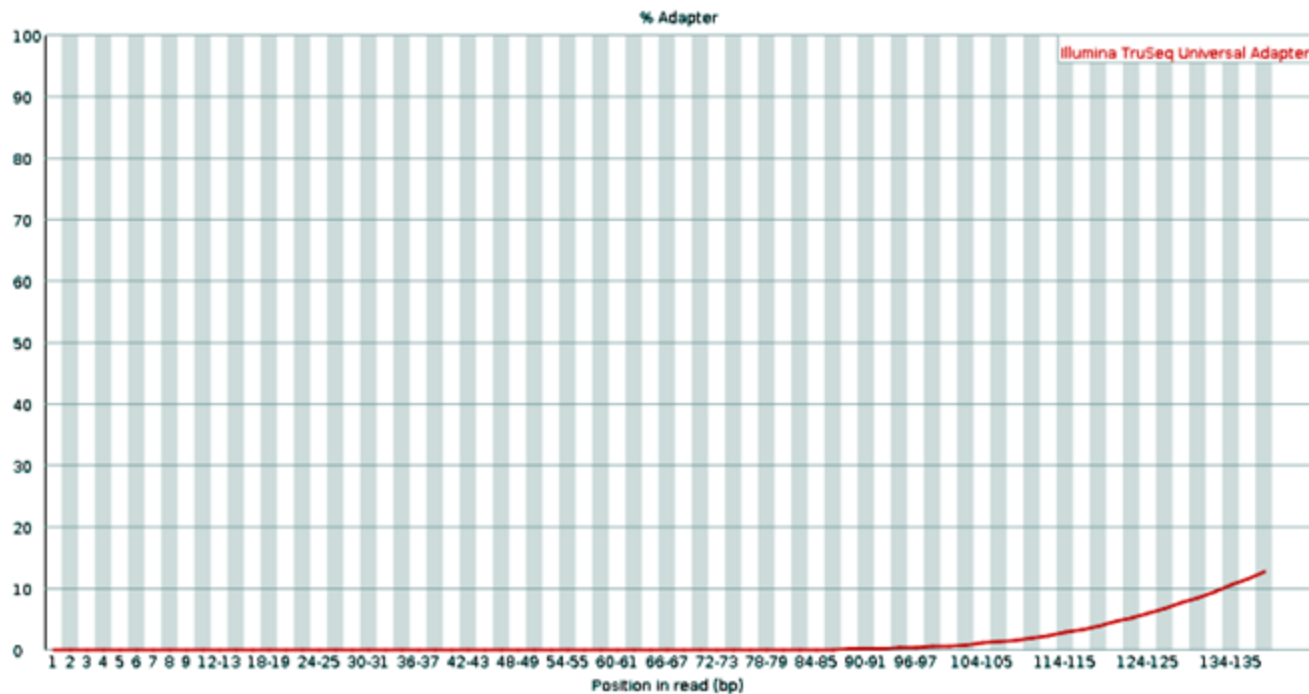


GOOD per base quality

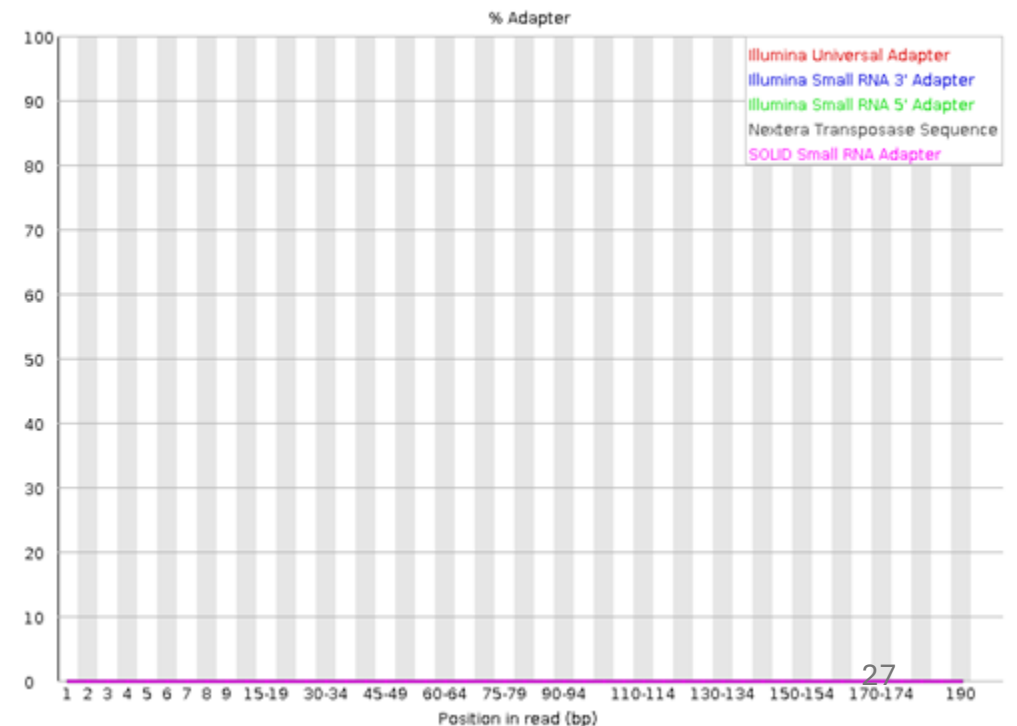


Adapter Content

- ❖ Cumulative plot of the fraction of reads where the sequence library adapter sequence is identified at the indicated base position.
- ❖ Only adapters specific to the library type are searched.



✓ Adapter Content



Trimming & Filtering Nanopore Reads



Finding and trimming adapter sequences from in ONT reads

Adapter can either be at:

- the end → trimmed off
- middle → indicate chimeric reads → split into separate reads

Takes time: aligns the reads in order to detect adapters

Supports demultiplexing for barcoded reads (Native, PCR, Rapid kits)

wdecoster/**nanofilt**

Filtering and trimming of long read sequencing data

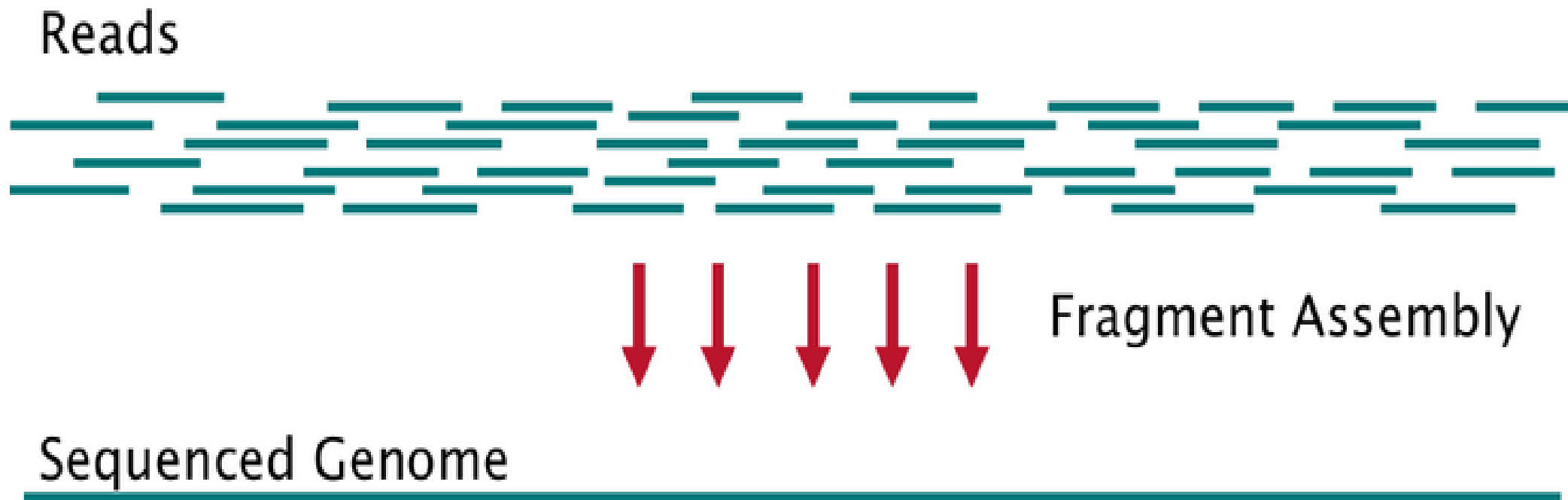


Nanofilt is a read filtering tool. It lets you:

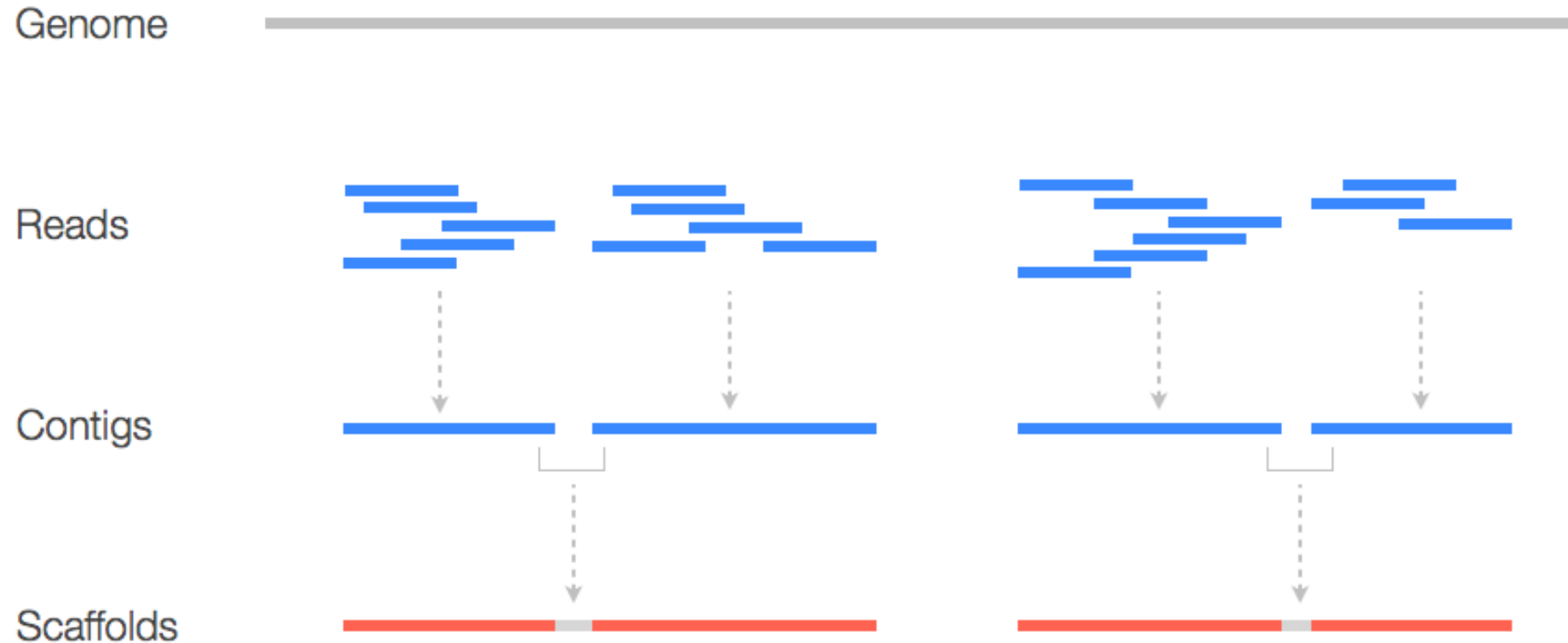
- Filter by **read length** (min/max)
- Filter by **quality score** (Q)

De Novo Assembly

Assembly in a perfect world

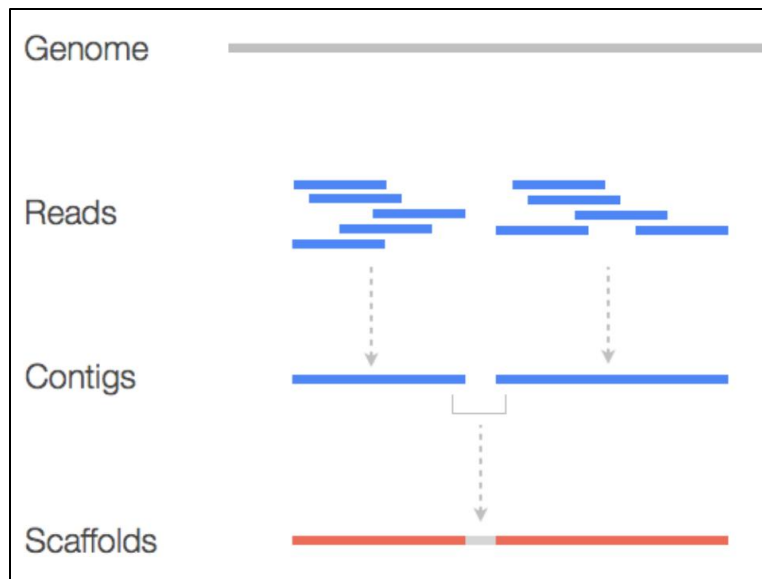
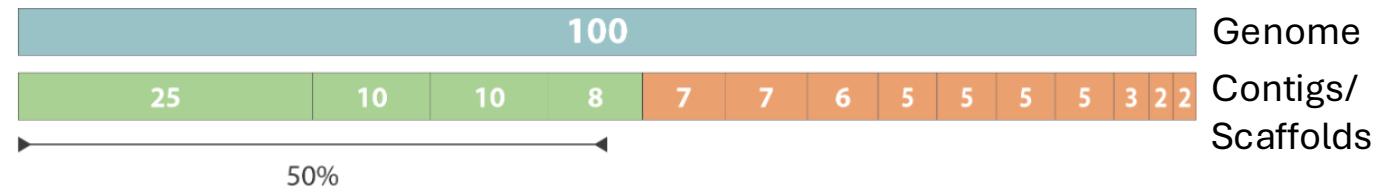


Assembly in real world



De-novo Assembly

- ❖ **Contigs:** continuous, gapless stretches of DNA sequence.
- ❖ **Scaffolds:** ordered arrays of contigs connected gaps to represent a more complete view of the genome



Genome size = 100 bp

Sort the contigs/scaffolds by decreasing size.

N50: length of the shortest contig/scaffold that covers at least 50% of the genome

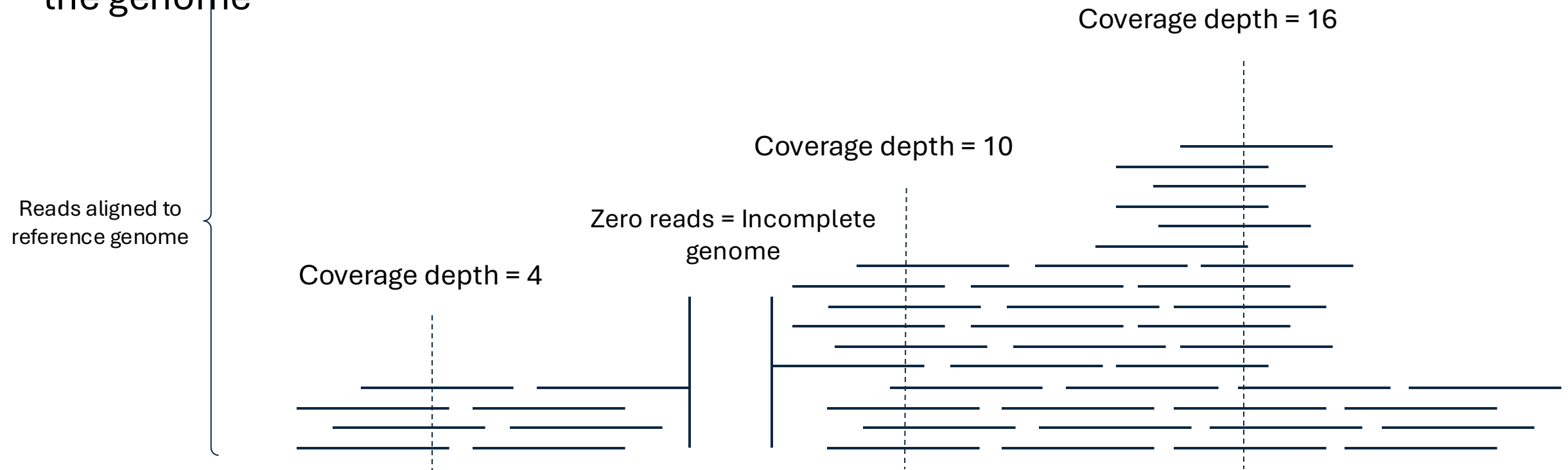
L50: Count until the smallest number of sequences that make 50% of genome

N50: 8 & L50: 4

Goal is to have high N50 and low L50.

Coverage depth/Read Depth

Coverage = How many times each base has been sequenced on average throughout the genome



De novo assembly tool

- Flye
- de Novo assembly = like rebuilding a shredded book from noisy strips (reads).
- Problem: repeated phrases (like “once upon a time”) confuse the reconstruction.
- Traditional assemblers (spades) need exact matches.
- Flye works with approximate matches (ONT prone to errors).
- Result: Flye builds contigs from unique regions, and shows repeats as unresolved if no read spans them fully.

Discussion

Metric	Sample A	Sample B	Sample C	Comments
Genome size (Mb)	3.8	3.8	3.8	All match the expected size
Coverage (×)	4	180	120	B best depth, C is okay but A we may have gaps
# Contigs	22,000	95	130	Fewer is better: B most contiguous, C moderate, A fragmented.
N50 (kb)	20	210	160	Longer contigs better: A very short contigs
L50	100	6	8	Highly fragmented
GC content (%)	60.5	41.5	41.5	Possible contamination on sample A

How To Determine the Number of Samples to Load

Example 1:

Organism: *Mycobacterium tuberculosis* (4.4 Mbp/4,400,000 bp)

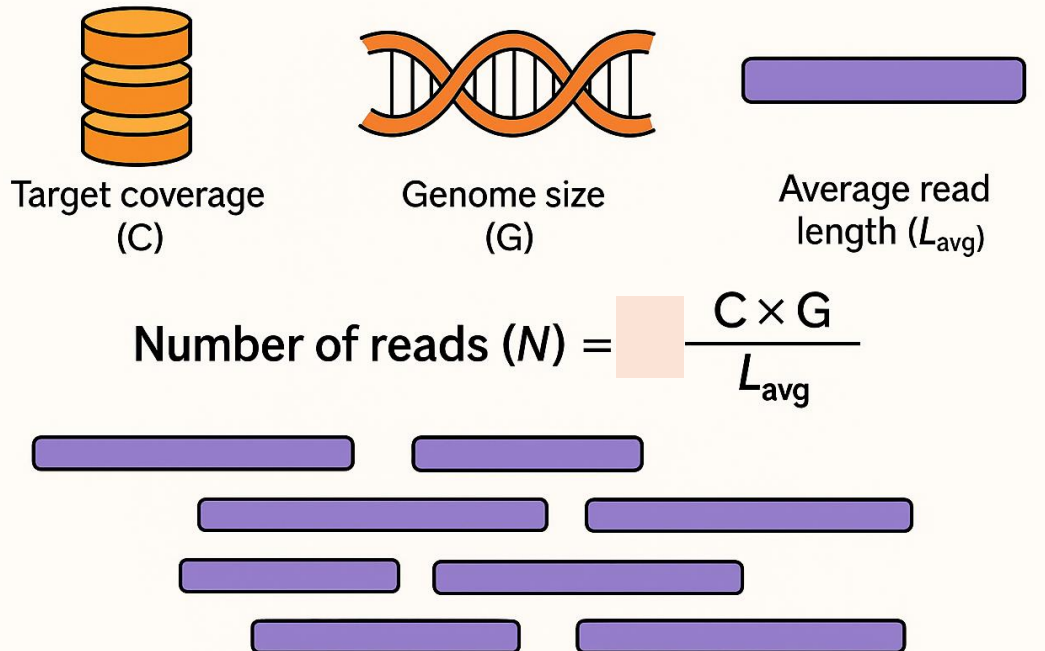
Coverage: 100 X

Bases per Sample (with 100X): 4.4 Mbp X 50 = 220 Mb

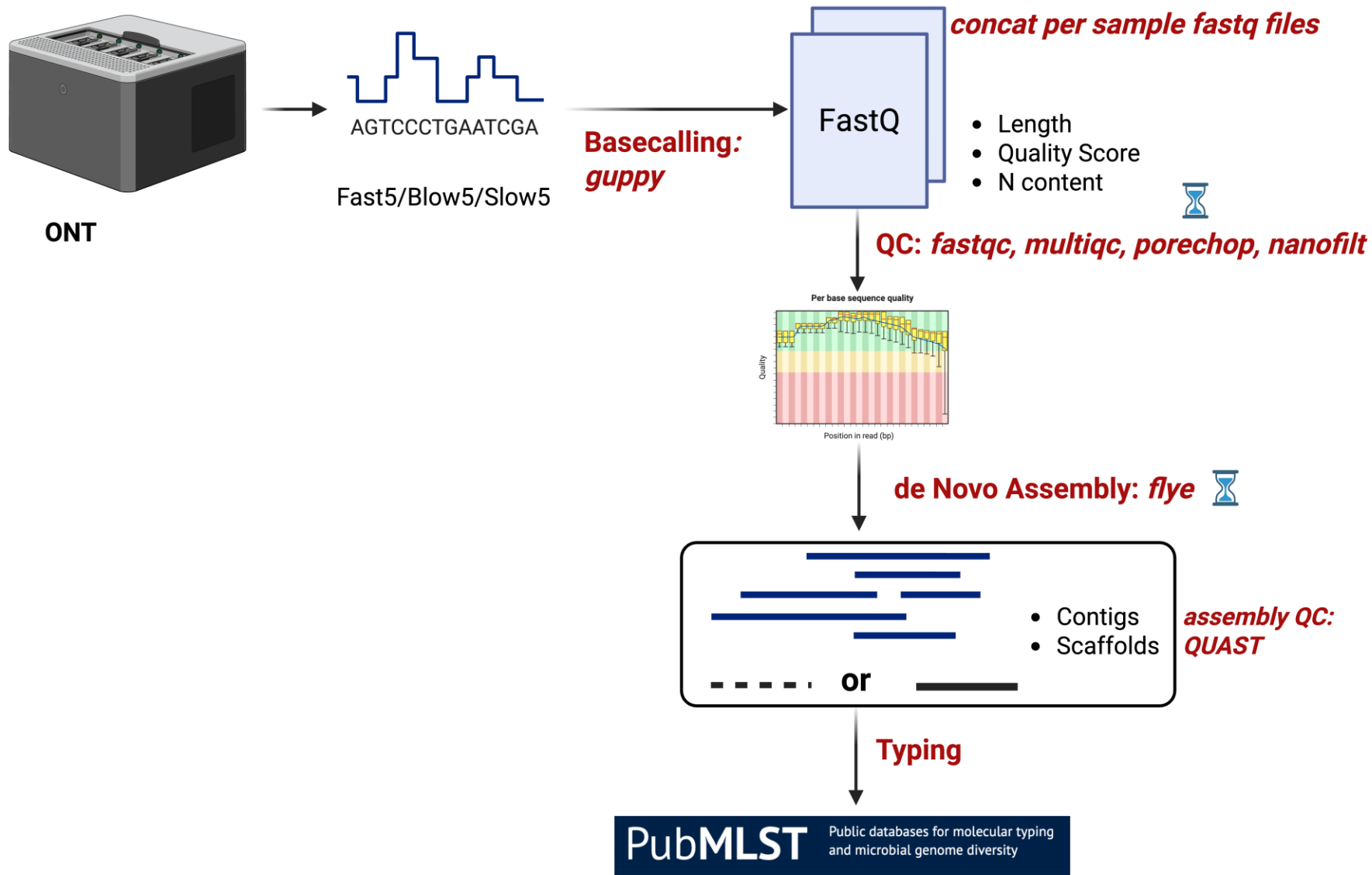
Average ONT Read Length: 5kbp/5,000bp

Reads per sample (5 kb reads): 220 Mb ÷ 5 kb ≈ 44,000 reads

How many TB samples will you load per run assuming a maximum of 5Gb & same read length (5kbp): 22 samples



Summary of the Pipeline



How to Run the Pipeline

Viewing the Bash Script

```
cat WCS-Assembly_v3.sh
```

Method 1 to know how to use the Script:

```
bash WCS-Assembly_v3.sh --help
```

OR

Method 2 to know how to use the Script

```
bash WCS-Assembly_v3.sh -h
```

Content of the Bash Script

ONT De Novo Assembly Pipeline (ONT-only)

=====

Description:

- Concatenate barcode FASTQs
- Adapter trimming (Porechop)
- QC (FastQC, MultiQC)
- Filtering (NanoFilt)
- Assembly (Flye)
- Evaluation (QUAST)

Supported Reads:

Only Oxford Nanopore Technologies (ONT) reads are supported.

Usage:

```
$(basename "$0") -i <input_dir> -o <output_dir> [-t <threads>] [--genome-size <size>] [--asm-coverage <cov>]  
$(basename "$0") --input <input_dir> --output <output_dir> [--threads <threads>]
```

Options:

-i, --input	Path to input directory containing barcode* folders	(required)
-o, --output	Path to output directory	(required)
-t, --threads	Number of threads to use (default: 4)	(optional)
-g, --genome-size	Estimated genome size for Flye (e.g. 5m, 3.2m, 500k)	(optional)
-c, --asm-coverage	Limit assembly coverage for Flye	(numeric, optional)
-h, --help	Show help and exit	

Running the Script

```
bash ont_denovo_assembly_fly.sh --input fastq_pass --output output/flye_assembly --threads 4
```

```
|— fastq_pass/
|   |— barcode01/
|   |   |— barcode01_0001.fastq.gz
|   |   |— barcode01_0002.fastq.gz
|   |   └ ...
|   |— barcode02/
|   |   |— barcode02_0001.fastq.gz
|   |   └ ...
|   |— unclassified/
|   |   └ unclassified_0001.fastq.gz
|   └ ...
|— fastq_fail/
|   |— barcode01/
|   |— barcode02/
|   └ unclassified/
```

```
🔍 Checking required tools...
✓ 'porechop' is installed.
✓ 'multiqc' is installed.
✓ 'flye' is installed.
✓ 'quast.py' is installed.
✓ 'NanoFilt' is installed.
✓ 'fastqc' is installed.
Found 1 barcode folders:
| — barcode04/
Combining chunks in barcode04...
```

Data analysis exercise

- ❖ 3 samples + 1 NC per group
- ❖ Script output: Fasta files
- ❖ Different Analysis Pipelines:

PubMLST



<https://tinyurl.com/2fc2t8wm> - to access the excel file to fill in the
PubMLST results

<https://tinyurl.com/yc33k7d4> - to access the fasta files

Data analysis exercise (PubMLST)

- What is the species
 - Tool: PubMLST Species ID
- Typing (Organism specific)
 - What is the ST type /CC / Finetyping?
 - Tool: *Neisseria* /Typing/ Strain typing
 - cgMLST loci recovered/closest match
 - Tool: *Neisseria* /Typing /cgMLST v3

	A	B	C	D	E	F	G	H	I	J	K
1						Nm					
2				Barcode	Species	MLST	PorA_VR1	PorA_VR2	FetA_VR	cgMLST v3 (% loci match)	closest isolates
3	Pool 1	Group1	WCS-1	1							
4			WCS-5	2							
5			WCS-9	3							
6			NTC	4							
7		Group2	WCS-2	5							
8			WCS-6	6							
9			WCS-10	7							
10			NTC	8							
11		Group3	WCS-3	9							
12			WCS-7	10							
13			WCS-11	11							
14			NTC	12							
15		Group4	WCS-4	13							
16			WCS-8	14							
17			WCS-12	15							
18			NTC	16							